

Detecting and Preventing DoS/DDoS Attacks through Monitoring and Limiting Malicious Traffic

Sneha Padhiar
U & P.U Patel Department of
Computer Engineering

Charusat University of Science and
Technology
Anand, India
snehapadhiar.ce@charusat.ac.in

Hetanshi Patel
U & P.U Patel Department of
Computer Engineering

Charusat University of Science and
Technology
Anand, India
22ce088@charusat.ac.in

Darshil Chocha
U & P.U Patel Department of
Computer Engineering

Charusat University of Science and
Technology
Anand, India
22ce021@charusat.ac.in

Abstract— As cyber threats continue to evolve, online systems are at significant danger from Denial-of-Service (DoS) and Distributed Denial-of-Service (DDoS) attack included disrupting services and disable to access for legitimate users. These attacks use to overwhelm the network by sending high volume of malicious request, resulting in server downtime and substantial operational costs. Dos attack, which is a single source that generates the disruptive traffic, while DDoS attacks uses multiple sources named handlers and zombies to overwhelm target, exploiting system resources and causing legitimate users to experience downtime. This paper introduces solution to mitigate and prevent DoS-DDoS attack by implementing real-time request monitoring, rate limiting and traffic redirection. In order to protect server resources for authorized users and lower the possibility of service interruption, the system redirect suspected users to a verification page when it detects unusual request rates, which are frequently an indication of DDoS activity. leveraging lightweight scripting in the Nginx framework to speed up response times and increase attack resilience, this approach offers a scalable and adaptable way to mitigate DoS/DDoS attacks. This paper shows that proactive rate limitation and traffic redirection offer a potential way to combat the increasing sophistication and frequency of DDoS attacks by improving real-time traffic control.

I. INTRODUCTION

Denial of service attacks poses a big threat to the small scale industry and also big scale industries as it targets the industry online services and application by flooding with unnecessary requests and number of this attacks [1] is shown in Fig. 1[2]. This overload consumes resources and blocks legitimate traffic. Distributed denial of service attacks poses a bigger threat as they uses multiple sources to launch large scale attacks, making the traditional security measures insufficient.

This paper focuses on a more immediate and practical defence mechanism: **real-time traffic analysis**. By detecting traffic anomalies in real-time, this system can identify and mitigate malicious traffic before significant damage occurs. We will also refer to industry-proven solutions like rate limiting, load balancing and geographical filtering, highlighting their role in a comprehensive defence strategy.

The below circuit (Fig. 2) shows the brief demo of dos attack.

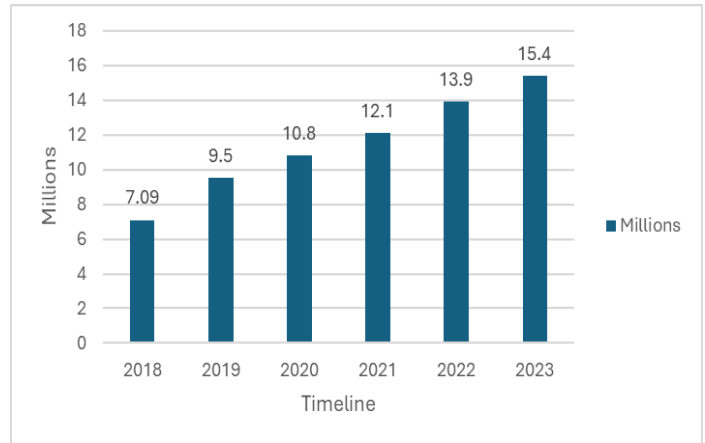


Fig.1 Global Trend of DDoS Attacks 2018-2023

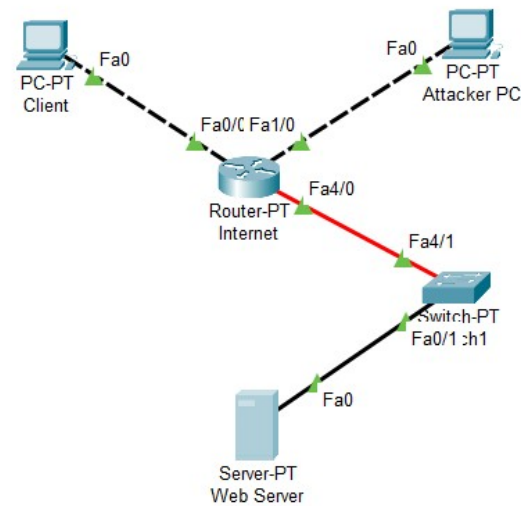


Fig. 2 Practically understanding of DoS

In Fig. 3 circuit assumes a scenario in which an attack is conducted from the attacker's personal computer to the web server where the client site is hosted. When the attacker starts sending the heavy loads of traffic to the web server then the server will start taking time to load the website because it is trying to full-fill the requests that is coming from the attacker's PC. Thus resulting into the slow access of the site to the real user(client) and also if heavy loads of requests continues then the site may stop working and genuine user will not able to use the service.

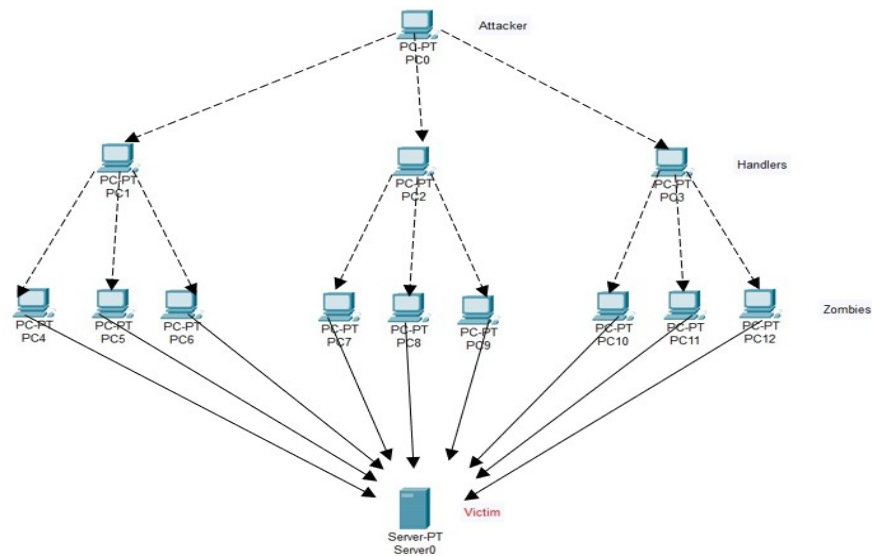


Fig.3 Practically understanding of DDoS[2]

In Distributed Denial of Service (DDoS) attack, attacker uses multiple devices, as handlers and zombies as shown in Fig. 3 to overwhelm victim machine with traffic by distribution method[3].

Here the attacker entity that initiates the attack and sends commands to handlers, that handlers receive commands from attacker and pass them to zombie machine and zombie machine performs actual attack by sending too many requests to target to overwhelm it, ultimately disrupting its service.

II. BACKGROUND OF DOS ATTACKS

A. Common Dos/DDoS Techniques

Denial-of-Service attacks overwhelm the target systems with heavy loads of traffic which result in the service not being accessible by legitimate users. Attackers exploit the network and application vulnerabilities to launch the attack. Common DoS techniques include[4][5]:

- **SYN Floods:** In this attack we flood the target with SYN requests. Too many requests which do not need acknowledgment thus without completing the TCP connection which results in exhausting the server resources with too many requests.
- **UDP Floods:** Attackers bombard the target with gigantic volumes of UDP packets, which lead to the network and server failure and leading to the service disruption.
- **HTTP Floods:** This technique overloads web servers with a heavy load of HTTP requests, overwhelming the application layer and causing service unavailability.

B. Existing Defence Strategies

DoS mitigation techniques to counter the advanced attack methods. Most used defence strategies include:

- **Load Balancing:** Distributing incoming traffic across multiple servers shown in Fig.4 thus

preventing a single server from getting overloaded ensuring the continuous running of all the servers under heavy loads.

```
(kali@kali)~$ lbd cyberbugs.in
lbd - load balancing detector 0.4 - Checks if a given domain uses load-balancing.
      Written by Stefan Behte (http://ge.mine.nu)
      Proof-of-concept! Might give false positives.

Checking for DNS-Loadbalancing: FOUND
cyberbugs.in has address 185.230.63.171
cyberbugs.in has address 185.230.63.186
cyberbugs.in has address 185.230.63.107

Checking for HTTP-Loadbalancing [Server]:
NOT FOUND

Checking for HTTP-Loadbalancing [Date]: , No date header found, skipping.

Checking for HTTP-Loadbalancing [Diff]: FOUND
< X-Seen-By: W1c2/pqHBqplxcWuFHckILxkNjrXdwdgtu6E0yACibU=
> X-Seen-By: 7U7NzZSqfMgd9YnjHLtUa7xkNjrXdwdgtu6E0yACibU=

cyberbugs.in does Load-balancing. Found via Methods: DNS HTTP[Diff]
```

Fig.4 Multiple servers

- **Geographical Traffic Filtering:** This technique blocks traffic from a specific region from where heavy loads of traffic are coming actively and which is suspicious of malicious activity, effectively mitigating geographically focused attacks.

C. Related Work

Here we describe about different DoS attacks based on Layer-4 and Layer-7.

1. TCP-flood Attack

Xinyu Yang et al. [6] proposed that TCP-flood attack can be done under IPv4 and IPv6. This attack misuses the mechanism of Transfer control Protocol.

Attacker exploits three-way handshake mechanism by only sending series of SYN packets shown in Fig.5 to victim machine/server with spoofed IP address. Victim

machine or server send in reply SYN-ACK packets, but after that attacker will not send any ACK packet and because of that connection remain partially open. This leads to fill the connection queue and memory buffer and legitimate users will not able to use the services.

2. ICMP(v6)-flood attack

ICMP based packet internet group(ping) uses echo response mechanism. In this attack, victim machine/server overloads with large number of packets which is send by attacker and because of those problems like resource wastage, network bandwidth drain and the legitimate users not able to get the services [7].

3. Incomplete HTTP requests using GET/POST/HEAD method

To communicate with web server client sends HTTP requests, In that client sends just ap part of HTTP header [8], never send the full header, it sends subsequent header at some time interval to keep socket alive. Here attacker sends multiple incomplete requests and exhaust the server's resources and because of that legitimate users will not able to get the services. This attack can be launched with minimal bandwidth [9].

Incomplete HTTP requests using POST/HEAD method is same as the GET method the only difference is it uses POST or HEAD method according to type of attack. In HEAD method server will not return message body in response[8], Because of that resource on attacker's side are saved and by the HEAD method, attacker target a page that is very expensive for server to create[9].

III. METHODOLOGY

This section describe implementation of real-time traffic analysis system for DoS attack

A. Traffic Monitoring

In wireshark we continuously monitors the incoming traffic by performing DoS attack using tool like hping3, Slowloris and LOIC. By monitor unusual traffic patterns DoS attack can be detected.

B. Signature Matching

If there are number of packets targeted on one ip from one or more Source addresses which are spoofed and no reply back that means no acknowledgement packet will captured then it can be DoS or DDoS attack.

hping3: There are large number of SYN packets per second will be captured and there is absence of SYN-ACK and ACK packets.

Slowloris: There are less packets compare to hping3 , connection may close eventually and leading to TCP (RST) flags appearing in the traffic.

LOIC: It completes three way TCP handshake. By sending connection request to server it will exhaust the resources.

C. Rate limiting

DDoS attack is difficult to detect as compare to DoS because it is perform by number of machines [10]and that can be in different geo location. In that case limiting no. of requests on site can be one of the solution to prevent this attack.

IV. EXPERIMENT SETUP AND RESULT

In this method we perform Dos attack using tool like Slowloris, LOIC and hping3 and done the analysis of different types of attack traffic. Here the target is DVWA and BWAPP we application for performing DoS attack on them in kali virtual machine and analysis of traffic done with the help of wireshark and implement rate limiting solution by deploying Nginx server with Lua-based script.

No.	Time	Source	Destination	Protocol	Length	Info
122	2.491661475	127.0.0.1	127.0.0.1	TCP	74	38322 → 80 [SYN] Seq=0
131	2.491789859	127.0.0.1	127.0.0.1	TCP	74	38330 → 80 [SYN] Seq=0
140	2.492176885	127.0.0.1	127.0.0.1	TCP	74	38346 → 80 [SYN] Seq=0
149	2.492277862	127.0.0.1	127.0.0.1	TCP	74	38358 → 80 [SYN] Seq=0
158	2.492433481	127.0.0.1	127.0.0.1	TCP	74	38374 → 80 [SYN] Seq=0
167	2.492489448	127.0.0.1	127.0.0.1	TCP	74	38380 → 80 [SYN] Seq=0
176	2.492598670	127.0.0.1	127.0.0.1	TCP	74	38388 → 80 [SYN] Seq=0
185	2.492713389	127.0.0.1	127.0.0.1	TCP	74	38398 → 80 [SYN] Seq=0
194	2.492852087	127.0.0.1	127.0.0.1	TCP	74	38412 → 80 [SYN] Seq=0
203	2.493008891	127.0.0.1	127.0.0.1	TCP	74	38428 → 80 [SYN] Seq=0
212	2.493139999	127.0.0.1	127.0.0.1	TCP	74	38432 → 80 [SYN] Seq=0
221	2.493242651	127.0.0.1	127.0.0.1	TCP	74	38442 → 80 [SYN] Seq=0
230	2.493395566	127.0.0.1	127.0.0.1	TCP	74	38444 → 80 [SYN] Seq=0
239	2.493498579	127.0.0.1	127.0.0.1	TCP	74	38452 → 80 [SYN] Seq=0
248	2.493666954	127.0.0.1	127.0.0.1	TCP	74	38456 → 80 [SYN] Seq=0
257	2.493882642	127.0.0.1	127.0.0.1	TCP	74	38468 → 80 [SYN] Seq=0
266	2.493982274	127.0.0.1	127.0.0.1	TCP	74	38488 → 80 [SYN] Seq=0
275	2.493983056	127.0.0.1	127.0.0.1	TCP	74	38488 → 80 [SYN] Seq=0
284	2.494139138	127.0.0.1	127.0.0.1	TCP	74	38496 → 80 [SYN] Seq=0
293	2.494198889	127.0.0.1	127.0.0.1	TCP	74	38510 → 80 [SYN] Seq=0
302	2.494748632	127.0.0.1	127.0.0.1	TCP	74	38526 → 80 [SYN] Seq=0
311	2.494927886	127.0.0.1	127.0.0.1	TCP	74	38536 → 80 [SYN] Seq=0

Fig.5 Slowloris pcap file

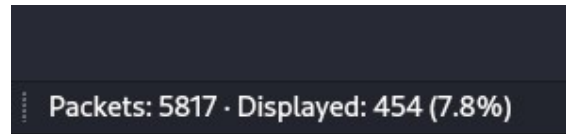


Fig.6 Number of packet sent for particular period of time

Slowloris: In Fig.5 we can see number of SYN packets in Fig.6 and Spikes on graph for particular interval as shown in Fig.7.

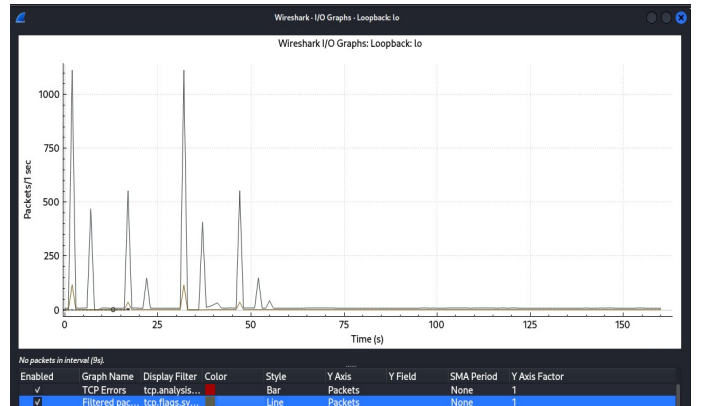


Fig.7 Graph of Packet per second vs time for slowloris

That means connection is incomplete and it was being opened and held for extended period with partial headers. In Fig.11 we can see packet A->B is 1 but the packet B->A is 0. That means no SYN-ACK and ACK packet was captured and connection remain open.

Hping: Here in hping(pcap file Fig.10) is same as Slowloris but number of packets sent per second is higher as shown in Fig.8 and Fig.9

Packets: 1207274 · Displayed: 1207274 (100.0%)

Fig.8 Number of packet for particular interval



Fig.9 Comparatively higher number of packets per seconds

No.	Time	Source	Destination	Protocol	Length	Info
1206...	9.685176621	127.0.0.1	127.0.0.1	TCP	54	42101 → 80 [RST] Seq=1 Win=0 Len=0
1206...	9.685182117	127.0.0.1	127.0.0.1	TCP	54	[TCP Port numbers reused] 42102 → 80 [SYN] Seq=0 Win=512 Len=0
1206...	9.685184557	127.0.0.1	127.0.0.1	TCP	58	80 → 42102 [SYN, ACK] Seq=0 Ack=1 Win=33280 Len=0 MSS=65495
1206...	9.685186240	127.0.0.1	127.0.0.1	TCP	54	42102 → 80 [RST] Seq=1 Win=0 Len=0
1206...	9.685190787	127.0.0.1	127.0.0.1	TCP	54	[TCP Port numbers reused] 42103 → 80 [SYN] Seq=0 Win=512 Len=0
1206...	9.685193106	127.0.0.1	127.0.0.1	TCP	58	80 → 42103 [SYN, ACK] Seq=0 Ack=1 Win=33280 Len=0 MSS=65495
1206...	9.685194466	127.0.0.1	127.0.0.1	TCP	54	42103 → 80 [RST] Seq=1 Win=0 Len=0
1206...	9.685198059	127.0.0.1	127.0.0.1	TCP	54	[TCP Port numbers reused] 42104 → 80 [SYN] Seq=0 Win=512 Len=0
1206...	9.685200250	127.0.0.1	127.0.0.1	TCP	58	80 → 42104 [SYN, ACK] Seq=0 Ack=1 Win=33280 Len=0 MSS=65495
1206...	9.685201839	127.0.0.1	127.0.0.1	TCP	54	42104 → 80 [RST] Seq=1 Win=0 Len=0
1206...	9.685205950	127.0.0.1	127.0.0.1	TCP	54	[TCP Port numbers reused] 42105 → 80 [SYN] Seq=0 Win=512 Len=0
1206...	9.685207972	127.0.0.1	127.0.0.1	TCP	58	80 → 42105 [SYN, ACK] Seq=0 Ack=1 Win=33280 Len=0 MSS=65495
1206...	9.685209233	127.0.0.1	127.0.0.1	TCP	54	42105 → 80 [RST] Seq=1 Win=0 Len=0
1206...	9.685212718	127.0.0.1	127.0.0.1	TCP	54	[TCP Port numbers reused] 42106 → 80 [SYN] Seq=0 Win=512 Len=0
1206...	9.685214735	127.0.0.1	127.0.0.1	TCP	58	80 → 42106 [SYN, ACK] Seq=0 Ack=1 Win=33280 Len=0 MSS=65495
1206...	9.685216094	127.0.0.1	127.0.0.1	TCP	54	42106 → 80 [RST] Seq=1 Win=0 Len=0
1206...	9.685219632	127.0.0.1	127.0.0.1	TCP	54	[TCP Port numbers reused] 42107 → 80 [SYN] Seq=0 Win=512 Len=0
1206...	9.685221583	127.0.0.1	127.0.0.1	TCP	58	80 → 42107 [SYN, ACK] Seq=0 Ack=1 Win=33280 Len=0 MSS=65495
1206...	9.685222802	127.0.0.1	127.0.0.1	TCP	54	42107 → 80 [RST] Seq=1 Win=0 Len=0

Fig.10 hping pcap file

Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Total Packets	Percent Filtered	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Re
127.0.0.1	38278	127.0.0.1	80	1	74 bytes	38	15	6.67%	1	74 bytes	0	0 bytes	2.49'
127.0.0.1	38294	127.0.0.1	80	1	74 bytes	39	15	6.67%	1	74 bytes	0	0 bytes	2.49'
127.0.0.1	38310	127.0.0.1	80	1	74 bytes	40	15	6.67%	1	74 bytes	0	0 bytes	2.49'
127.0.0.1	38322	127.0.0.1	80	1	74 bytes	41	15	6.67%	1	74 bytes	0	0 bytes	2.49'
127.0.0.1	38330	127.0.0.1	80	1	74 bytes	42	15	6.67%	1	74 bytes	0	0 bytes	2.49'
127.0.0.1	38340	127.0.0.1	80	1	74 bytes	166	15	6.67%	1	74 bytes	0	0 bytes	17.51'
127.0.0.1	38346	127.0.0.1	80	1	74 bytes	43	15	6.67%	1	74 bytes	0	0 bytes	2.49'
127.0.0.1	38346	127.0.0.1	80	1	74 bytes	167	15	6.67%	1	74 bytes	0	0 bytes	17.51'
127.0.0.1	38348	127.0.0.1	80	1	74 bytes	168	15	6.67%	1	74 bytes	0	0 bytes	17.51'
127.0.0.1	38352	127.0.0.1	80	1	74 bytes	169	15	6.67%	1	74 bytes	0	0 bytes	17.51'
127.0.0.1	38356	127.0.0.1	80	1	74 bytes	170	15	6.67%	1	74 bytes	0	0 bytes	17.51'
127.0.0.1	38358	127.0.0.1	80	1	74 bytes	44	15	6.67%	1	74 bytes	0	0 bytes	2.49'
127.0.0.1	38358	127.0.0.1	80	1	74 bytes	171	15	6.67%	1	74 bytes	0	0 bytes	17.51'

Fig.11 Packet was not sent from B to A

Ethernet · 1		IPv4 · 1	IPv6 · 1	TCP · 18	UDP							
Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration
127.0.0.1	80	127.0.0.1	38454	100,306	42 MB	0	49,988	3 MB	50,318	39 MB	0.000000	17.1748
127.0.0.1	80	127.0.0.1	38450	114,513	43 MB	3	57,103	4 MB	57,410	39 MB	0.029322	17.1540
127.0.0.1	33162	127.0.0.1	46061	856	57 kB	11	428	29 kB	428	28 kB	16.028610	7.4568
127.0.0.1	38382	127.0.0.1	80	103,447	40 MB	5	51,855	36 MB	51,592	3 MB	0.030293	17.1542
127.0.0.1	38384	127.0.0.1	80	105,494	43 MB	6	52,910	40 MB	52,584	3 MB	0.034198	17.1503
127.0.0.1	38402	127.0.0.1	80	102,020	45 MB	1	51,168	41 MB	50,852	3 MB	0.002157	17.1832
127.0.0.1	38410	127.0.0.1	80	95,777	41 MB	7	48,047	38 MB	47,730	3 MB	0.037273	17.1276
127.0.0.1	38412	127.0.0.1	80	101,769	43 MB	4	51,027	39 MB	50,742	3 MB	0.030171	17.1483
127.0.0.1	38424	127.0.0.1	80	104,020	42 MB	9	52,151	39 MB	51,869	3 MB	0.075188	17.1128
127.0.0.1	38436	127.0.0.1	80	97,477	40 MB	8	48,874	37 MB	48,603	3 MB	0.066133	17.1223
127.0.0.1	38464	127.0.0.1	80	115,184	42 MB	2	57,722	38 MB	57,462	4 MB	0.002538	17.1813
127.0.0.1	36382	::1	80	8 904 bytes	16	5	512 bytes	3	392 bytes	24.030941	0.0008	
127.0.0.1	36396	::1	80	9 1 kB	17	5	524 bytes	4	478 bytes	25.032543	0.0029	
127.0.0.1	59512	::1	80	8 904 bytes	12	5	512 bytes	3	392 bytes	19.793447	0.0010	
127.0.0.1	59516	::1	80	8 904 bytes	13	5	512 bytes	3	392 bytes	20.982460	0.0005	
127.0.0.1	59526	::1	80	8 904 bytes	14	5	512 bytes	3	392 bytes	21.983685	0.0011	
127.0.0.1	59534	::1	80	9 1 kB	15	5	524 bytes	4	478 bytes	22.992230	0.0012	
127.0.0.1	60802	::1	80	8 904 bytes	10	5	512 bytes	3	392 bytes	0.537934	0.0175	

Fig. 12 Mostly complete three way TCP handshake

No.	Time	Source	Destination	Protocol	Length	Info
1839..	17.173158544	127.0.0.1	127.0.0.1	TCP	66	80 -> 38382 [ACK] Seq=1 Ack=
1839..	17.173164988	127.0.0.1	127.0.0.1	TCP	98	38382 -> 80 [PSH, ACK] Seq=
1839..	17.173168876	127.0.0.1	127.0.0.1	TCP	66	80 -> 38382 [ACK] Seq=1 Ack=
1839..	17.173174475	127.0.0.1	127.0.0.1	TCP	98	38382 -> 80 [PSH, ACK] Seq=
1839..	17.173179264	127.0.0.1	127.0.0.1	TCP	66	80 -> 38382 [ACK] Seq=1 Ack=
1839..	17.173184857	127.0.0.1	127.0.0.1	TCP	98	38382 -> 80 [PSH, ACK] Seq=
1839..	17.173189651	127.0.0.1	127.0.0.1	TCP	66	80 -> 38382 [ACK] Seq=1 Ack=
1839..	17.173195324	127.0.0.1	127.0.0.1	TCP	98	38382 -> 80 [PSH, ACK] Seq=
1839..	17.173200159	127.0.0.1	127.0.0.1	TCP	66	80 -> 38382 [ACK] Seq=1 Ack=
1839..	17.173205714	127.0.0.1	127.0.0.1	TCP	98	38382 -> 80 [PSH, ACK] Seq=
1839..	17.173210479	127.0.0.1	127.0.0.1	TCP	66	80 -> 38382 [ACK] Seq=1 Ack=
1839..	17.173215972	127.0.0.1	127.0.0.1	TCP	98	38382 -> 80 [PSH, ACK] Seq=
1839..	17.173220789	127.0.0.1	127.0.0.1	TCP	66	80 -> 38382 [ACK] Seq=1 Ack=
1839..	17.173226382	127.0.0.1	127.0.0.1	TCP	98	38382 -> 80 [PSH, ACK] Seq=
1839..	17.173231145	127.0.0.1	127.0.0.1	TCP	66	80 -> 38382 [ACK] Seq=1 Ack=
1839..	17.173236661	127.0.0.1	127.0.0.1	TCP	98	38382 -> 80 [PSH, ACK] Seq=
1839..	17.173241452	127.0.0.1	127.0.0.1	TCP	66	80 -> 38382 [ACK] Seq=1 Ack=
1839..	17.173247124	127.0.0.1	127.0.0.1	TCP	98	38382 -> 80 [PSH, ACK] Seq=
1839..	17.173251885	127.0.0.1	127.0.0.1	TCP	66	80 -> 38382 [ACK] Seq=1 Ack=
1839..	17.173257432	127.0.0.1	127.0.0.1	TCP	98	38382 -> 80 [PSH, ACK] Seq=
1839..	17.173262266	127.0.0.1	127.0.0.1	TCP	66	80 -> 38382 [ACK] Seq=1 Ack=
1839..	17.173267784	127.0.0.1	127.0.0.1	TCP	98	38382 -> 80 [PSH, ACK] Seq=

Frame 1: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 80, Dst Port: 38454, Seq: 1, Ack: 1, Len: 0

Fig.13 LOIC pcap file

LOIC: By LOIC we can send TCP/UDP/HTTP request to overwhelmed the server by completing three way TCP hand shake as Fig.12 and Fig.13 with the high number of requests within short timeframe, as shown in Fig.14 and Fig.15

Downloaded	Requested	Failed
	5129421	

Fig.14 TCP/UDP/HTTP request

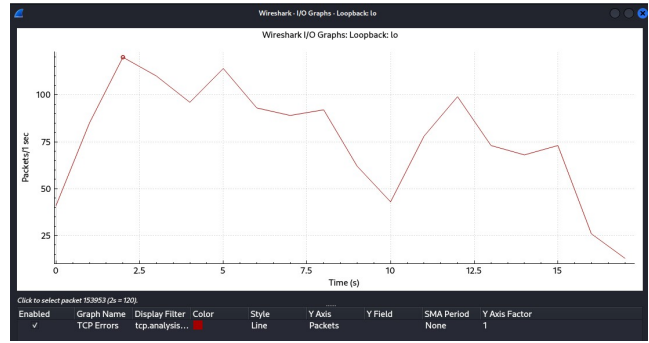


Fig.15 Graph of Packet per second vs time for LOIC

Lua-based script that dynamically limits incoming requests to stop DDoS attacks. This environment consists of two main components : the server and client-side script. The server, which is configured to detect and take action on excessive requests using rate limiting, and client side script that sends high volume of requests to start DDoS attack. By this setup, the goal is to observe how the server responded to legitimate versus malicious traffic, Once a specific request threshold has been reached, suspected bots are redirected to a verification page.

The Lua script embedded within Nginx utilized shared dictionary to monitor IP addresses and tally request frequencies, with a timer configured to automatically reduce the count over time, upon exceeding the request threshold, the server would promptly redirect the client to a challenge page, so neutralizing the DDoS attempt as shown in Fig.16.

```
(kali@kali)~$ tail -f /var/log/nginx/access.log
127.0.0.1 - - [12/Nov/2024:01:42:56 +0530] "GET /challenge HTTP/1.1" 200 2139 "-" "curl/8.7.1" limit_status=-
127.0.0.1 - - [12/Nov/2024:01:42:56 +0530] "GET / HTTP/1.1" 302 138 "-" "curl/8.7.1" limit_status=PASSED
127.0.0.1 - - [12/Nov/2024:01:42:56 +0530] "GET /challenge HTTP/1.1" 200 2139 "-" "curl/8.7.1" limit_status=-
127.0.0.1 - - [12/Nov/2024:01:42:56 +0530] "GET / HTTP/1.1" 302 138 "-" "curl/8.7.1" limit_status=PASSED
127.0.0.1 - - [12/Nov/2024:01:42:56 +0530] "GET /challenge HTTP/1.1" 200 2139 "-" "curl/8.7.1" limit_status=-
127.0.0.1 - - [12/Nov/2024:01:43:00 +0530] "GET / HTTP/1.1" 302 138 "http://localhost/challenge" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0" limit_status=PASSED
127.0.0.1 - - [12/Nov/2024:01:43:00 +0530] "GET /challenge HTTP/1.1" 200 856 "http://localhost/challenge" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0" limit_status=-
127.0.0.1 - - [12/Nov/2024:01:43:06 +0530] "GET /challenge HTTP/1.1" 200 856 "http://localhost/challenge" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0" limit_status=-
127.0.0.1 - - [12/Nov/2024:01:43:13 +0530] "GET / HTTP/1.1" 302 138 "http://localhost/challenge" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0" limit_status=PASSED
127.0.0.1 - - [12/Nov/2024:01:43:13 +0530] "GET /challenge HTTP/1.1" 200 856 "http://localhost/challenge" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0" limit_status=-
127.0.0.1 - - [12/Nov/2024:01:45:10 +0530] "GET / HTTP/1.1" 200 3990 "-" "curl/8.7.1" limit_status=PASSED
127.0.0.1 - - [12/Nov/2024:01:45:10 +0530] "GET / HTTP/1.1" 200 3990 "-" "curl/8.7.1" limit_status=PASSED
127.0.0.1 - - [12/Nov/2024:01:45:10 +0530] "GET / HTTP/1.1" 200 3990 "-" "curl/8.7.1" limit_status=PASSED
127.0.0.1 - - [12/Nov/2024:01:45:10 +0530] "GET / HTTP/1.1" 200 3990 "-" "curl/8.7.1" limit_status=PASSED
127.0.0.1 - - [12/Nov/2024:01:45:10 +0530] "GET / HTTP/1.1" 200 3990 "-" "curl/8.7.1" limit_status=PASSED
127.0.0.1 - - [12/Nov/2024:01:45:10 +0530] "GET / HTTP/1.1" 302 138 "-" "curl/8.7.1" limit_status=PASSED
```

Fig. 16 Rate Limiting Response in Logs

V. CONCLUSION

This research shows how to integrate Lua scripting into the Nginx framework to effectively fight against DoS and DDoS attacks. It also shows that real-time traffic analysis and adaptive rate limitation are feasible methods for mitigating such attacks. Tested in simulated attack scenarios, the system demonstrated the ability to identify and redirect high-frequency malicious traffic while maintaining accessibility for authorized users. This approach uses dynamic request monitoring to make sure servers can handle unexpected spikes in traffic, which are common in DDoS

situations, without suffering from downtime or performance issues. By including other elements like IP reputation and geographic filtering, further research might improve this method even more and offer a more comprehensive DDoS mitigation strategy.

REFERENCES

- Donkal, G. and Donkal, A., 2021. Digital Twin and the Detection and Location of DoS Attacks to Secure Cyber-Physical UAS. In *Digital Twin Technology* (pp. 135-163). CRC Press.

2. Cisco Annual Internet Report (2018–2023) White Paper, (accessed June 11, 2020). [Online]. Available: <https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html>
3. Abubakar, R., Aldegheishem, A., Majeed, M.F., Mehmood, A., Maryam, H., Alrajeh, N.A., Maple, C. and Jawad, M., 2020. An effective mechanism to mitigate real-time DDoS attack. *IEEE Access*, 8, pp.126215-126227.
4. Dimolianis, M., Pavlidis, A. and Maglaris, V., 2021. Signature-based traffic classification and mitigation for DDoS attacks using programmable network data planes. *IEEE Access*, 9, pp.113061-113076.
5. Sumantra, I. and Gandhi, S.I., 2020, July. DDoS attack detection and mitigation in software defined networks. In *2020 International Conference on System, Computation, Automation and Networking (ICSCAN)* (pp. 1-5). IEEE.
6. Xinyu Yang, Ting Ma and Yi Shi, "Typical DoS/DDoS Threats under IPv6", ICCGI '07, Proceedings of the International Multi-Conference on Computing in the Global Information Technology
7. Jon Postel, Transmission Control Protocol, RFC793, Internet Engineering Task Force, 1981
8. R. Fielding, J. Gettys, J. Mogul, H. Frystyk, L. Masinter, P. Leach and T. Berners-Lee, "Hypertext Transfer Protocol -- HTTP/1.1", RFC2616, Internet Engineering Task Force, June 1999
9. N. Tripathi, "DoS and DDoS Attacks: Impact, Analysis and Countermeasures."
10. Özçelik, İ. and Brooks, R., 2020. *Distributed denial of service attacks: Real-world detection and mitigation*. CRC Press.
11. Sathupadi, K., 2023. Ai-based intrusion detection and ddos mitigation in fog computing: Addressing security threats in decentralized systems. *Sage Science Review of Applied Machine Learning*, 6(11), pp.44-58.
12. Al Harits, S. and Zy, A.T., 2024. Securing Servers From DDoS Attacks Using NGINX. *Novice Research Exploration*, 1(2).
13. Neto, L.V., Nogueira, V., de Moura, A.L. and Ierusalimschy, R., 2020. Linux Network Scripting with Lua. In *Netdev 0x14, THE Technical Conference on Linux Networking*. <https://netdevconf.info/0x14/pub/papers/22/0x14-paper22-talk-paper.pdf>.
14. Tripathy, S.S. and Behera, B., 2024. EVALUATION OF FUTURE PERSPECTIVES ON SNORT AND WIRESHARK AS TOOLS AND TECHNIQUES FOR INTRUSION DETECTION SYSTEM. *EVALUATION*, 53(10).
15. Deshpande, D., 2023, December. Improving the DDoS Protection and AI Systems Using Wireshark. In *2023 16th International Conference on Developments in eSystems Engineering (DeSE)* (pp. 234-239). IEEE.